rogii-wellbore-geology-prediction

[7th Place Solution] HMM + UNet (agent is all you need)

Rank	7
Team	roglike
Author	Gaopeng Ren
Topic ID	733154
Version	Original

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733154
Retrieved with Kaggle CLI on 2026-08-06.

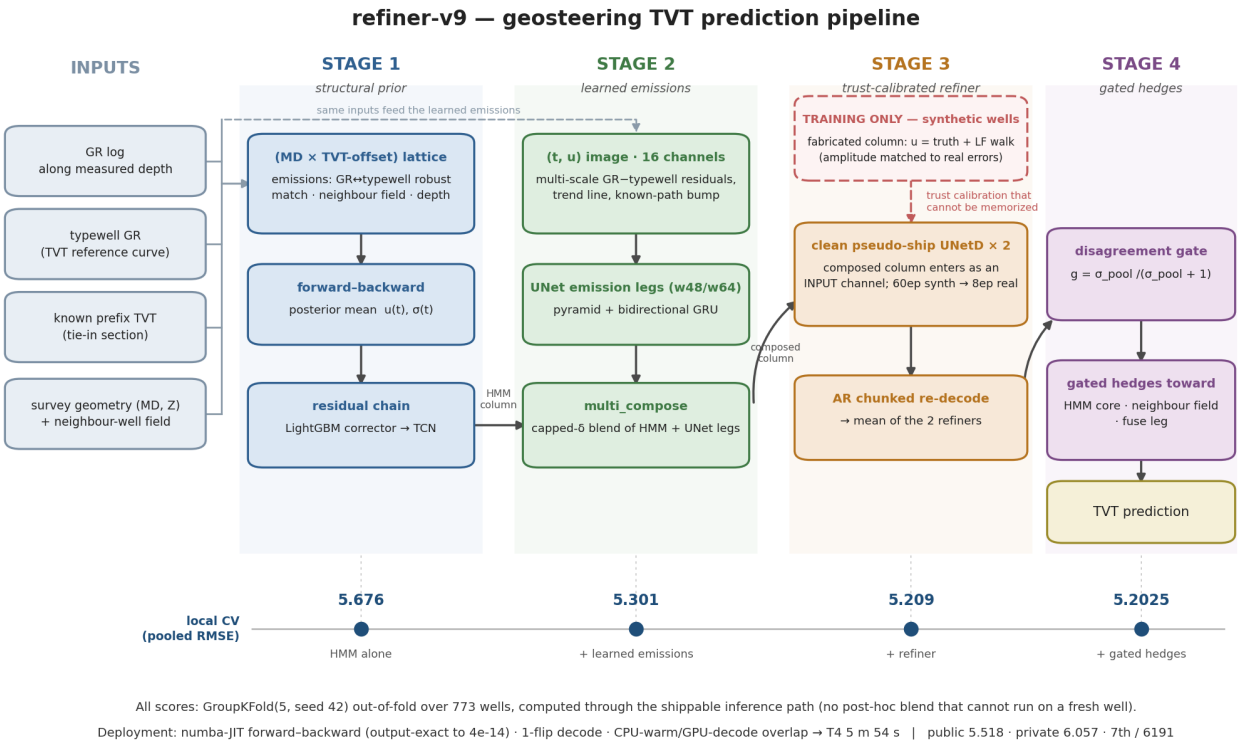
I and and my teammate (@ **ejixxx**) thank the hosts and everyone who contributed to public discussions and code. It is a tough but interesting journey in this competition. Without any geology knowledge, we are excited that agents can help us achieve this place. We did not even know the details or how our solution worked in the end. What we mainly did was to stop agents from cheating and point the potential resources to agents to obtain new ideas. The final results show that most of our recent solutions can achieve gold medals, which is impressive that agents can provide such a solid solution. What we learned from this competition is not only that agents are very powerful, but we also realised several mistakes that agents might make and how we alleviate them. The typical mistakes are:

1. **Cheat with data leakage**. For example, borrow the bundle from public code, and blend it with our local solution without realising that the splits might be different. After this, we will ask agents many times to ensure their local improvements.
2. **Give up very quickly**. Agents, especially Claude agents, since they usually copy context during the conversation, making them get more and more confirmed once they think the goal cannot be achieved. The only solution for this is to start a new session. If you are unsure about the details and just tried to give general ideas like try UNet, Transformer, and etc, would be better to do several rounds of verification and confirm with agents. For example, we already tried UNet earlier in our 7-8 public LB solution and they didn't show much improvements on the public LB. When in the 5-6 public LB region, UNet helped a lot.
3. **Agents cannot devise new ideas**. Agents are good at replicating but not at innovation. We need to explicitly prohibit agents from infinitely small perturbations like meaningless hyperparameter tuning.
4. **Agents need resources provided by humans**. We find that the `websearch` and literature search of Claude agents are not very smart. They usually use similar, long, and specific keywords for the search, which makes the search results very narrow. We alleviate this problem by directly specifying the resources, the keywords, and papers that can help agents get more insights.

One thing we are not satisfied with is that we did not develop a unified agent workflow for the Kaggle competition. One harness (e.g., AutoResearch/Tree-based AI Scientists) is not enough to obtain a very high place, but we believe there should be a method/workflow that can help agents achieve very high performance without humans involved. Anyway, the above is our write-up for how to use agents to build our solutions; the following are the explanations for the solution built by agents.

---

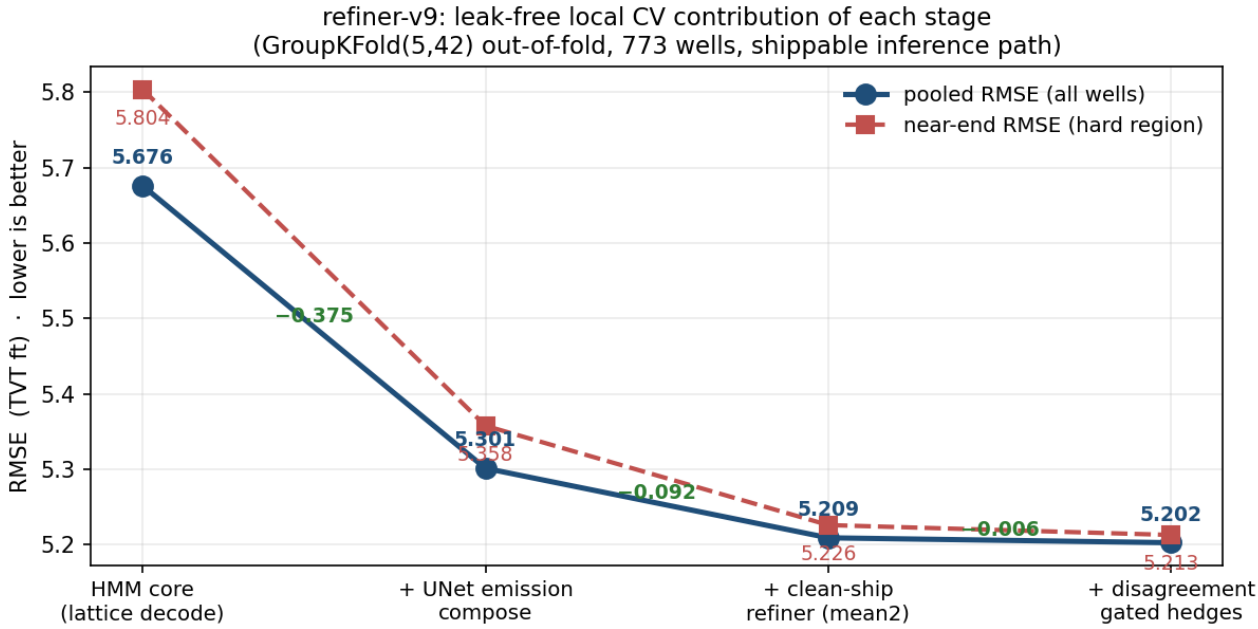
1. The pipeline



All scores: GroupKFold(5, seed 42) out-of-fold over 773 wells, computed through the shippable inference path (no post-hoc blend that cannot run on a fresh well).

Deployment: numba-JIT forward-backward (output-exact to 4e-14) · 1-flip decode · CPU-warm/GPU-decode overlap → T4 5 m 54 s | public 5.518 · private 6.057 · 7th / 6191

Every stage is scored the same way: **GroupKFold(5, seed 42) out-of-fold over all 773 training wells, through the shippable inference path** — the model that runs on a fresh unseen well, never a post-hoc blend that couldn't be reconstructed at inference.



The HMM alone is 5.676. Learned emissions take it to 5.301. The refiner stage — the part that took the longest to get right — takes it to 5.209, and the gated hedges close at **5.2025 pooled / 5.2128 near-end**.

2. The idea that made stage 3 work: pretrain on fabricated mistakes

The refiner is a second-pass UNet that sees the first-pass prediction as an extra input channel and is asked to fix it. The obvious way to train it — finetune on real wells with the out-of-fold column as input — fails, and the diagnosis is the interesting part:

> A fold net decodes **its own training wells at RMSE 3.20** but held-out wells at **9.34**.

So during training the conditioning channel is nearly perfect, and the cheapest way to cut the loss is to copy it. The net never learns when the input is wrong, because in its training distribution it almost never is. Trust calibration is destroyed by memorization. The fix: **learn trust on wells that cannot be memorized.** We generate ~6000 synthetic wells and fabricate the conditioning channel:

```
u_ship = truth + amplitude × low-passed random walk        (anchored at the tie-in)
amplitude ~ lognormal(median ≈ 2.9, σ = 0.68)              (matches the measured
15% of wells drawn as "good ship" (amplitude 0.3–1.0)       real error distribution)
```

Pretrain 60 epochs on these, then finetune only 8 epochs on real wells. Now the net has seen thousands of wells where the input column is confidently wrong in realistic ways, and it has learned where to distrust it — a skill that survives the short real-data finetune. The pretrained state is reusable across conditioning generations, which made the whole later campaign cheap (10 min per refiner variant). v9 ships the plain version of this: **two such refiners, averaged.**

---

3. The gated-signal principle

Several auxiliary signals looked worthless: the neighbour-well field prediction correlates with the composed column's residual at r ≈ −0.02. Blending it in flat does nothing. But the two refiners disagree precisely where they are unsure. Using their per-node standard deviation as a gate, `g = σ/(σ+1)`, and applying the hedges only through that gate:

```
p ← mean(refiners)
p ← p + 0.10·g·(HMM core − p)          shrink toward the structural prior
p ← p + 0.15·g·clip(neighbour − p)     the "dead" signal, revived
p ← p + 0.15·g·clip(fuse leg − p)
p ← p + 0.05·g·clip(neighbour − p)
```

Each stage is worth little alone; gated, together they are worth 0.006 pooled and 0.013 on the hard near-end wells. The general lesson: a signal that is useless on average can be useful exactly where your ensemble admits it is lost.

---

4. Why it is called "wall-survival"

Every submission we made in the preceding 24 hours **scored nothing — they all hit the 9-hour runtime wall.** The hidden set is much larger than the 3-well stub the notebook sees, so the binding metric is seconds-per-well, not stub time. v9 is the version that was engineered to survive:

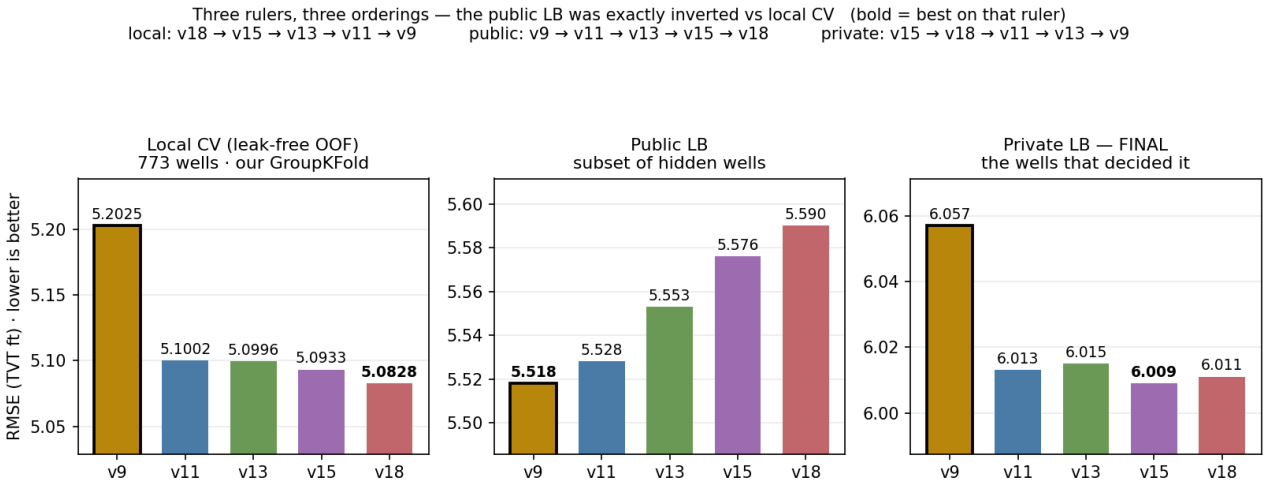
- **numba-JIT forward-backward** for the HMM: banded tridiagonal transition kernel, unique-emission-row exponentials, float64 running state. 25 s → 12 s per decode, **output-exact to 4e-14** (verified against the numpy path before shipping).
- **1-flip decode** on the main and fuse legs (drop the flip-TTA — a real accuracy cost, taken deliberately).
- **CPU-warm / GPU-decode overlap**: a prefetch thread decodes the first wells' UNet legs while the forkserver workers are still warming.
- fp16 autocast with NaN guards, per-family decode chunking, batched pip installs, `gc.disable()` around model load.

Result: **T4 stub 5 m 54 s, ≈50–65 s per hidden well** — enough margin for a hidden set several times larger than anything we had evidence for. It scored; the fancier variants that came later were built on top of this runtime foundation.

---

5. The honest postscript: which ruler was telling the truth

We finished the competition with five constructions of increasing sophistication. They disagree about which is best, and which ruler you ask decides the answer:



- **Local CV said**: v18 best, v9 clearly worst (5.083 vs 5.203).
- **Public LB said**: exactly the opposite ordering — v9 best (5.518), v18 worst (5.590).
- **Private LB said**: local CV was basically right after all — v9 is last (6.057), the enriched constructions cluster at 6.009–6.015.

We chose v9 because of the public LB, reasoning that the extra pool enrichment in the later versions was fitting the out-of-fold error structure and not transferring. A paired bootstrap over our 773 wells said the public inversion had only a 1.75% chance of being sampling noise — so we trusted it.

That reasoning was wrong in an instructive way. The public split was small enough to produce a fully inverted ranking, and our bootstrap measured our well distribution, not the public split's. The leak-free local CV had the ordering right the whole time.

It didn't cost us: the private spread across all five was 0.048, and the rank is identical for any of them — v9 held 7th place. But the takeaway for the next competition is clear: **a rigorously leak-free local CV over 773 wells beats a public score computed on a fraction of that, and small public-LB differences are not evidence.** The one thing the public LB did establish reliably was the constant offset (local + ≈0.31), not the ordering within a 0.1 band.